

C programming

Trainer : Rajiv K

Email : rajiv.kamune@sunbeaminfo.com

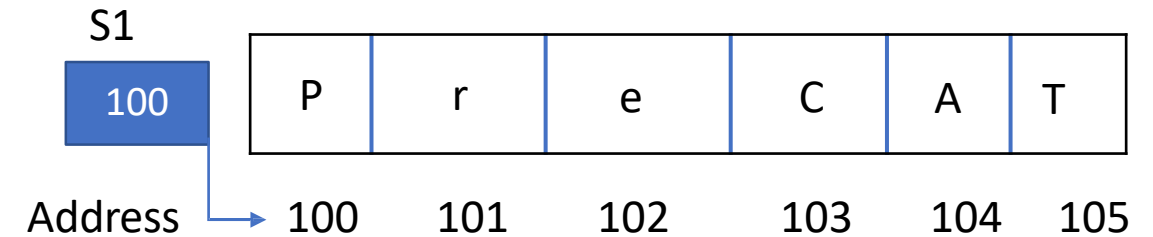


Strings and character arrays :

- Character Array :

Collection of character elements.

```
char s1[5] = {'P','r','e','C','A','T'}; //not a string
```



- String:

Collection of character elements with sentinel element '\0'

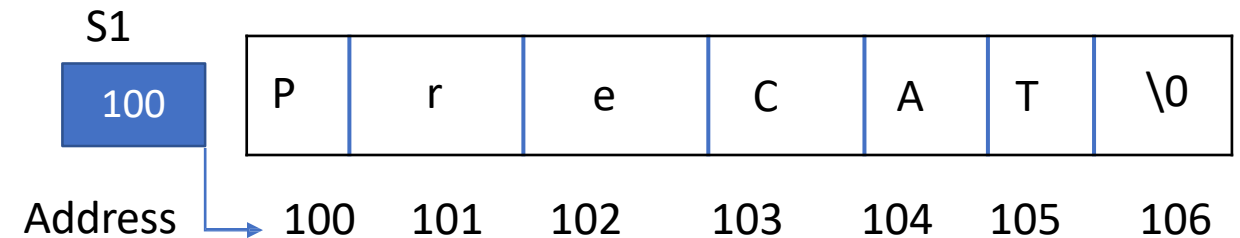
```
char s1[] = {'P','r','e','C','A','T','\0'} //string
```

```
;
```

```
char s1[] = "PreCAT";
```

- Size :

```
char *s1 = "PreCAT";
```



Always need to reserve 1 byte extra for sentinel element NULL.



String :

- Not a primitive datatype.
- C compiler provides special library function to handle strings.
- These library functions are declared in string.h
- e.g.
- Strlen
- strcpy
- strcmp
- strcat
- strstr
- strupr
- strlwr
- strrev
- strchr



String :

- Example :

```
char arr[5] = "abcde";  
int j;  
for(j=0; j<5; j++)  
    printf("%c",arr[j]);
```

- Accepting string as a input

```
char str[20];  
scanf("%s",str); // input  
printf("%s",str); // output
```

```
char str[20];  
gets(str);  
puts(str);
```



String scan sets :

- %s
- %[^\n]s //scan upto \n (single line)
- %[^.]s // scan upto . (multiple line)
- %[0-9]s// scan upto digits
- %[^0-9]s// scan upto alphabets
- %[A-Z]s// scan upto capital
- %[^a-z]s// scan upto capital
- %[^A-Z]s// scan upto small letter
- %[a-z]s// scan upto small letter



Command Line Arguments :

- The C language provides a method to pass parameters to the `main()` function.
- This is typically accomplished by specifying arguments on the operating system command line.
- The prototype for `main()` looks like:
 - `int main(int argc, char *argv[]) { ... }`
 - The first parameter is the number of items on the command line (`int argc`).
 - `argc` always retains the count of arguments passed to `main`
 - Each argument on the command line is separated by one or more spaces, and the operating system places each argument directly into its own null-terminated string.
 - Note: The name of the program is counted and is the first value.
 - Note: Values are defined by lists of characters separated by whitespace.
 - The second parameter passed to `main()` is an array of pointers to the character strings containing each argument (`char *argv[]`).
 - `Argv` catches actual arguments passed at command prompt to `main` function
 - Note: The array has a length defined by the `number_of_args` parameter.
- If we add **`char **env`** an argument to `main` it will display list of environment variables.
- Environment variables are used for information about your home directory, terminal type, and so on; you can define additional variables for other purposes. The set of all environment variables that have values is collectively known as the *environment*.



Standard main() prototypes :

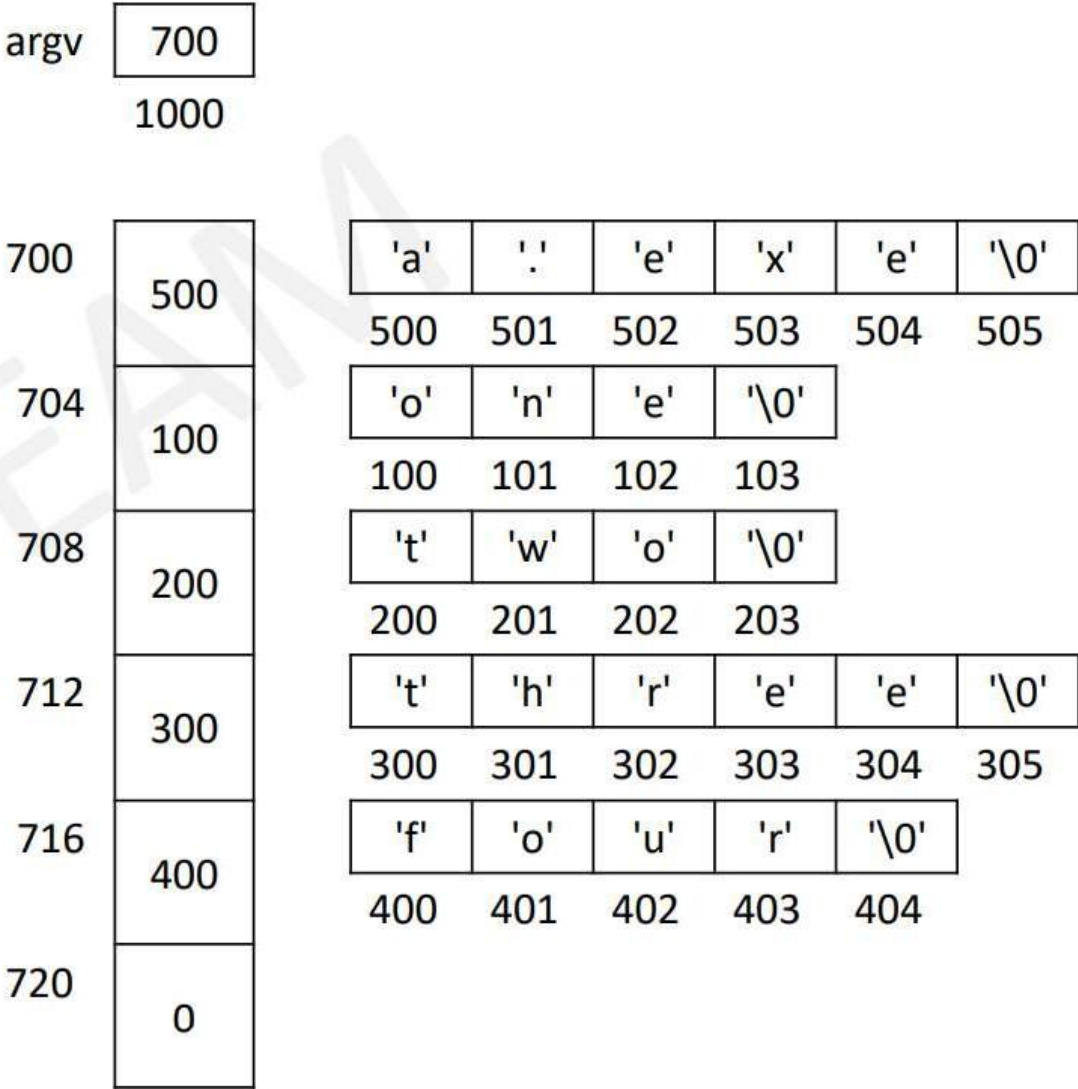
- `int main();`
- `int main(int argc, char *argv[]);`
- `int main(int argc, char*argv[], char*env[]);`
- `argc` represents number of arguments passed to program when it is executed from command line.
- `argv` represents argument vector or argument values.
- `envp` represents system information.



Command Line Arguments :

- Command line arguments are information passed to the program while executing it on command line.
- cmd> a.exe one two three four

```
int main(int argc, char *argv[]) {  
    int i;  
    for(i=0; i < argc; i++)  
        puts(argv[i]);  
    return 0;  
}
```



2-D Arrays :

- Arrays that we have considered till now are one dimensional arrays, a single line of elements.
- Often data comes naturally in the form of a table, e.g., spreadsheet, which need a two-dimensional array.
- Two-dimensional (2D) arrays are indexed by two subscripts, one for the row and one for the column.
- Example: `int a[2][3];`

Logically it may be viewed as a two-dimensional collection of data, two rows and three columns each location is of type `int`.

Syntax Declaration :-

```
datatype varName[rowSize][colSize]={element initialization values};
```

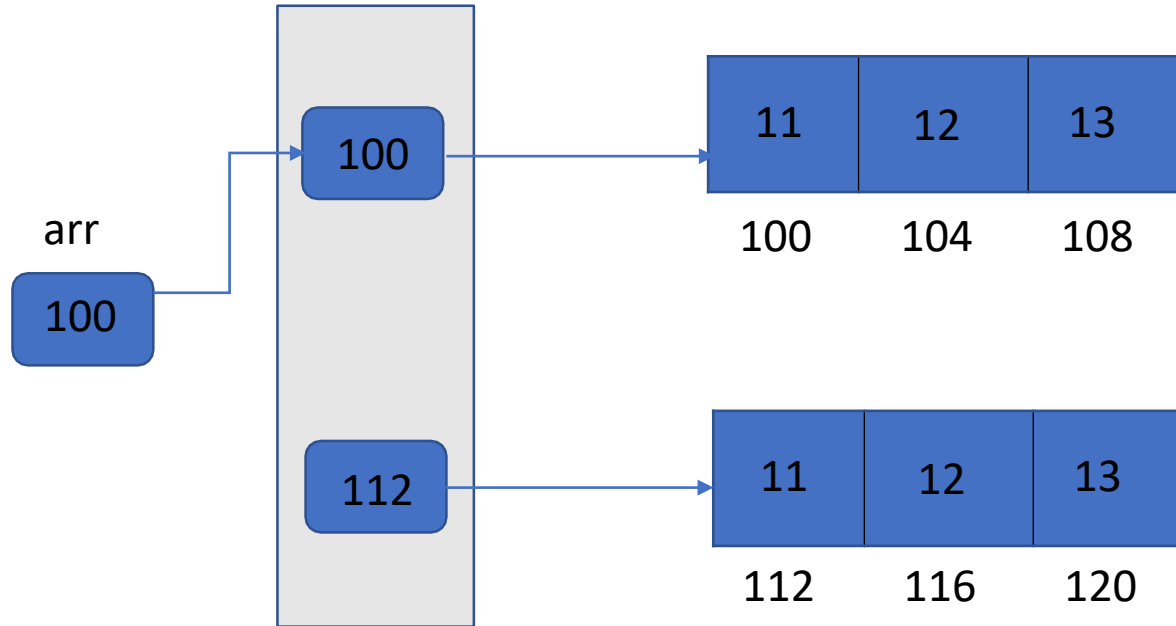
```
int arr[2][3] = {{11,22,33},{44,55}};
```

```
int arr[2][3] = {11,22,33,44,55};
```



Multi Dimensional Array :

- `int arr[2][3] = {{11,22,33},{44,55,66}};`



`arr[1][1] == *(*arr+1)+1`

`arr = 100`

Address of row / pointer to int
`arr+1 = 112`

Address of row / pointer to int
`*(*arr+1) = 112`

Address of int
`*(*arr+1)+1 = 116`

Address of int
`*(*(*arr+1)+1) = 55`



2D array Declarations :

- **Valid Declarations :**

- 1.int mat[2][2]={{1,1},{1,2},{2,1},{2,2}}; //allowed
- 2.int mat1[ROW][COL]={{1,1},{1,2},{2,1},{2,2}}; //allowed
- 3.int mat3[][COL]={{1,1},{1,2},{2,1},{2,2}}; // allowed
- 4.int mat4[2][2];

- **Invalid Declarations :**

- 1.int mat[][]={{1,1},{1,2},{2,1},{2,2}}; // not allowed
- 2.int mat2[ROW][]={{1,1},{1,2},{2,1},{2,2}}; //not allowed



Thank You!!

